

CryptoChain Analyzer Dashboard

Hash Functions and Blockchain — Final Report

Cryptography | Universidad Alfonso X el Sabio | Prof. Jorge Calvo | 2025–26

Student	Ariel Núñez Valencia
GitHub	Arielnv77
Repository	blockchain-dashboard-Arielnv77
Deadline	14 May 2026
Grade	0–4 points

1. Cryptographic Metrics — Implementation and Meaning

1.1 SHA-256 Double Hash and Proof of Work

Bitcoin's Proof of Work requires miners to find a nonce such that $\text{SHA-256}(\text{SHA-256}(\text{header}))$ produces a hash below a target threshold. This is implemented manually in `m2_block_header.py` using only Python's `hashlib` library, without any external cryptographic dependencies:

```
def double_sha256(data: bytes) -> bytes:
    h1 = hashlib.sha256(data).digest()
    return hashlib.sha256(h1).digest()

hash_display = double_sha256(header_bytes)[::-1].hex() # Bitcoin
byte_order
valid = int(hash_display, 16) <= target # PoW check
```

The byte reversal is required because Bitcoin stores hashes in little-endian display order. The dashboard shows the full verification result — VALID or INVALID — along with the number of leading zero bits in the hash.

1.2 The bits Field and the 256-bit Target Threshold

Each block header contains a 4-byte compact representation of the PoW target called bits. The dashboard decodes this field into the full 256-bit integer target using the following formula:

```
exp = bits >> 24 # exponent byte
mant = bits & 0xffffffff # mantissa (3 bytes)
target = mant * (1 << (8 * (exp - 3))) # full 256-bit target
```

Difficulty is then computed as the ratio of the genesis target to the current target:

```
MAX_TARGET = 0x00000000FFFF0000...0000 # genesis block (bits = 0x1d00ffff)
difficulty = MAX_TARGET / target
```

As of block 947,132 (April 2026), the difficulty is approximately 135 T (trillion), meaning each hash attempt has a probability of roughly 2^{-80} of succeeding. This is visualised in the dashboard as the number of leading zero bits required in a valid block hash.

1.3 Leading Zero Bits

The number of leading zero bits in a valid block hash directly quantifies the PoW difficulty. A valid hash must be numerically smaller than the target, which is equivalent to having at least k leading zero bits where $k = 256 - \text{floor}(\log_2(\text{target}))$. This is computed as:

```
leading_zeros = 256 - int(hash_hex, 16).bit_length()
```

Current mainnet value: approximately 80 leading zero bits, corresponding to a per-attempt success probability of 2^{-80} .

1.4 Inter-block Time Distribution (M1)

Under steady network conditions, Bitcoin block inter-arrival times follow an exponential distribution with parameter $\lambda = 1/600$ s (target: one block every 10 minutes). This follows from the memoryless property of the Poisson process underlying the mining race. The M1 module fetches the last 100 blocks, computes consecutive timestamp differences, and plots the empirical distribution alongside the theoretical $\text{Exp}(\lambda=1/600 \text{ s})$ curve. The observed mean inter-arrival time was 728 s — slightly above the 600 s target — reflecting recent difficulty underestimation before the next retarget epoch.

1.5 Difficulty History and Retarget Mechanism (M3)

Every 2,016 blocks (approximately two weeks), Bitcoin automatically adjusts difficulty using the formula:

```
new_difficulty = old_difficulty * (2016 * 600) / actual_time_seconds
```

The M3 module retrieves the last 20 retarget epochs from the Blockchain.info API and plots difficulty evolution over time. Each retarget event is marked on the chart. The ratio between actual block time and the 600 s target is also displayed, showing whether miners were faster or slower than expected during each epoch.

1.6 Block Header Structure (M2)

The 80-byte Bitcoin block header is parsed from its raw hexadecimal representation using Python's struct module with little-endian byte order:

Field	Bytes	Description
version	0–3	Block version (little-endian int32)
prev_hash	4–35	Hash of previous block header (reversed)
merkle_root	36–67	Merkle root of all transactions (reversed)
timestamp	68–71	Unix timestamp (little-endian uint32)
bits	72–75	Compact target encoding (little-endian uint32)
nonce	76–79	Mining nonce (little-endian uint32)

2. AI Component — Anomaly Detector

2.1 Model Selection and Justification

The AI component implements a statistical anomaly detector for Bitcoin inter-block times. The model choice is justified by the theoretical properties of the mining process: under normal conditions, block inter-arrival times follow $\text{Exp}(\lambda = 1/600 \text{ s})$. Deviations from this distribution can indicate mining pool synchronisation events, network propagation delays, or unusual miner behaviour.

A Z-score approach on log-transformed times was chosen over direct Z-scoring because the raw exponential distribution is heavily right-skewed. Log-transformation normalises the distribution, making Z-score thresholds meaningful:

```
log_t = np.log(block_times)          # log-transform
z      = scipy.stats.zscore(log_t, ddof=1)
anomaly = np.abs(z) > 3               # flag outliers beyond 3 sigma
```

2.2 Training Data

The model operates on live data fetched from the Blockstream API. The last 100 confirmed blocks are retrieved at each dashboard refresh cycle (every 60 seconds). No offline training dataset is required — the exponential baseline is defined analytically from the protocol specification ($\lambda = 1/600 \text{ s}$), and the Z-score parameters (mean, std) are estimated from the current window of 100 blocks.

2.3 Evaluation Metrics

Metric	Value (sample)	Interpretation
Anomaly rate	8.0 %	Fraction of blocks flagged as anomalous ($ Z > 3$ on log T)
Mean interval	728.9 s	Observed average vs 600 s target
Std deviation	622.2 s	Spread of inter-arrival times
KS p-value	0.0025	Kolmogorov-Smirnov test vs $\text{Exp}(1/600)$
Fast blocks (< 60 s)	7	Unusually rapid blocks
Slow blocks (> 1800 s)	8	Unusually delayed blocks

The Kolmogorov-Smirnov test compares the empirical inter-arrival time distribution against the theoretical $\text{Exp}(\lambda=1/600 \text{ s})$ baseline. A low p-value (0.0025) indicates the observed data deviates from the theoretical baseline — consistent with the fact that the current epoch's mean interval (728 s) is above the 600 s target, suggesting the difficulty was set slightly too high relative to actual hash rate during this period.

2.4 Limitations

- The 100-block window is small; a larger historical window would improve anomaly detection reliability.
- The Z-score threshold of 3 is a heuristic — no ground truth labels exist for anomalous Bitcoin blocks.
- The model does not distinguish between anomalies caused by network issues versus intentional mining pool behaviour.

3. External References

The following sources were consulted during the implementation of this project:

[1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Available at: <https://bitcoin.org/bitcoin.pdf>

[2] **Blockstream API Documentation**. REST API for Bitcoin mainnet block and transaction data. <https://blockstream.info/api>

[3] **Blockchain.info API**. Aggregated Bitcoin network statistics and difficulty history. <https://blockchain.info/api>

[4] **Bitcoin Wiki — Difficulty**. Technical specification of the difficulty adjustment algorithm. <https://en.bitcoin.it/wiki/Difficulty>

[5] **Bitcoin Wiki — Block hashing algorithm**. Specification of the 80-byte block header and SHA-256d computation. https://en.bitcoin.it/wiki/Block_hashing_algorithm

4. Optional Module M5 — Merkle Proof Verifier

Module M5 verifies the inclusion of any Bitcoin transaction in a block using a cryptographic Merkle proof. Every Bitcoin block header stores the Merkle root of all its transactions (bytes 36–67, little-endian). The module fetches all transaction IDs for a block via the Blockstream API (GET /block/{hash}/txids), rebuilds the full binary Merkle tree using SHA-256d exclusively from Python's hashlib, extracts the sibling-hash proof path for any selected transaction index, and verifies the proof by re-hashing the path and comparing against the stored merkle_root. All byte-order conventions follow the Bitcoin protocol: txids are reversed (internal little-endian order) before hashing, and the final computed root is reversed back to display order for comparison. If the number of nodes at any level is odd, the last node is duplicated — matching the protocol specification exactly.

```
def _sha256d_pair(a: bytes, b: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(a + b).digest()).digest()
level = [bytes.fromhex(txid)[::-1] for txid in txids] # display → internal order
if len(level) % 2 == 1: level.append(level[-1]) # duplicate last if odd
```

The proof is verified by iterating through the sibling hashes, combining each with the running hash in the correct order (left/right determined by whether the node index is even or odd), and confirming the final result equals the header's merkle_root field. This proves transaction inclusion without downloading all block transactions — a key scalability property of Bitcoin's SPV (Simplified Payment Verification) design.

5. Optional Module M6 — 51% Attack Cost Estimation

Module M6 estimates the cost in USD/hour of executing a 51% attack on the Bitcoin network based on live hash rate data. The attacker must acquire more than half the current network hash rate (~970 EH/s), which at current ASIC efficiency (Antminer S21 Pro: 234 TH/s at 3510 W) would require approximately 4.14 million mining units consuming 14.5 GW of power.

At an industrial electricity cost of \$0.06/kWh (the value used in m6_security_score.py), the operational cost alone exceeds \$873,000 per hour, not counting capital expenditure on hardware (estimated at \$20.7 billion at \$5,000/unit, matching the ASIC_PRICE_USD constant in the module). This analysis is consistent with Section 11 of the Nakamoto whitepaper, which shows that attack probability decreases exponentially with confirmation depth.
